Denver, CO | U.S. Citizen
(217) 994-1229
thiggs911@gmail.com

Tyler Higgs

Software Engineer

Portfolio
LinkedIn
GitHub

Education

Massachusetts Institute of Technology, Computer Science **Sep 2018 — Jun 2022**

Experience

Software Engineer (Offstream) **Dec 2025 — Jun 2026**
Next.js, React, TypeScript, Tailwind, Vercel, Prisma, GraphQL, PostgreSQL, REST hybrid

- Scoped and shipped an **AI-powered form builder** for structured data capture, end-to-end
- Designed and implemented a **public-facing REST API** for logging carbon removal events across multiple event types
- Built **record locking** and **data export** for in-review submissions, preserving audit integrity and enabling external verification
- Improved **dashboard performance by 8x** by optimizing data fetching and rendering
- Built an AI-powered answer recommendation system that surfaces suggested responses for new submissions by matching previously accepted registry answers and users' prior answers across differently structured custom forms

Software Engineer (Patterson) **Jul 2023 — Aug 2025**
TypeScript, Angular, C#, .NET, Entity Core, SQL, Azure, REST, Docker, Terraform remote

- Lead engineer on my team's **AngularJS→Angular migration**; pioneered best practices across teams and authored the migration docs that cut new-contributor setup time
- Developed and maintained a form-building microservice using **Nx workspaces**, **modern Angular**, and **C#, .NET, Azure**
- Served as **accessibility** and **component testing** lead, applying **WCAG** best practices across new and existing components; discovered and fixed SQL injection vulnerabilities

Software Engineer (Vividly; previously Software Engineering Intern, Jun — Aug 2021) **Jul 2022 — Jun 2023**
React, TypeScript, GraphQL, Python, Django, PostgreSQL, Docker remote

- Modernized legacy stack by introducing Django, GraphQL, and TypeScript React, achieving **10x faster load times** and improved type safety
- Migrated and cleansed **>1 TB** of customer data from Firebase to PostgreSQL using custom ETL pipelines, yielding faster queries and stronger consistency
- Designed and implemented an **end-to-end encryption** system for a data scraping tool in **Python** and **TypeScript**, securing sensitive customer data and meeting compliance requirements
- Built test suites spanning **React Testing Library**, GraphQL integration tests, **PyTest**, and **Playwright**; as an intern, cut CI costs 50% by moving GitHub Actions to self-hosted **GCP** runners

Solo Projects

Eave — Home Inspection SaaS (Nuxt, Vue, SwiftUI, SQLite, Stripe, Claude API, Cloudflare, SSR, iOS)

- Built and shipped a solo SaaS web + iOS product for home inspectors: custom templates, on-site report entry, and generated **PDF** and shareable web reports
- Cut migration friction from competitors with an **AI template import** pipeline (**Claude** + **Zod** structured outputs) converting Spectora/HomeGauge CSV, Excel, and PDF exports into editable templates over **streaming SSE**
- Integrated **Stripe Connect** (direct charges, card + **ACH**); modeled asynchronous settlement as a unit-tested state machine with **idempotent**, out-of-order-safe webhooks
- Native **SwiftUI** iOS app captures data offline via an on-device outbox that queues edits and photos and syncs on reconnect; deployed to **Cloudflare** edge with **SSR**, D1, **CI/CD**, and **Playwright** E2E tests

Extended Markdown Editor (Nuxt, Vue, OAuth, SSR, Tailwind, SQLite, TypeScript, CI/CD) **GitHub**

- Built an **SSR** Nuxt markdown editor with Google **OAuth**, SQLite autosave, and SEO-friendly server-rendered pages
- Implemented a custom $O(n)$ markdown parser supporting tabs, accordions, icons, and inline components, with **Go compiled to WebAssembly** for faster syntax highlighting

Collaborative Sudoku (Angular (zoneless), TypeScript, Rust, Node, Firebase, Playwright, IndexedDB) **GitHub**

- Added **collaborative mode** with real-time puzzle updates and chat using **event-driven architecture** and **optimistic updates**; verified **eventual consistency** with **Playwright** tests simulating concurrent users
- Built a **Rust** puzzle generator with unit tests and a **Node** API publishing to **Firebase**; Angular client persists offline via IndexedDB